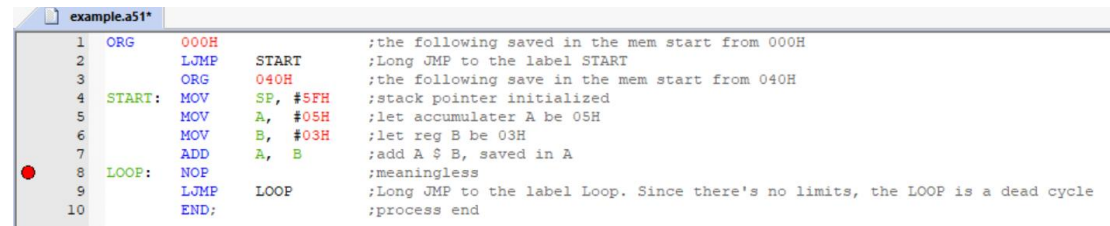


嵌入式作业一

1. Keil 的使用入门

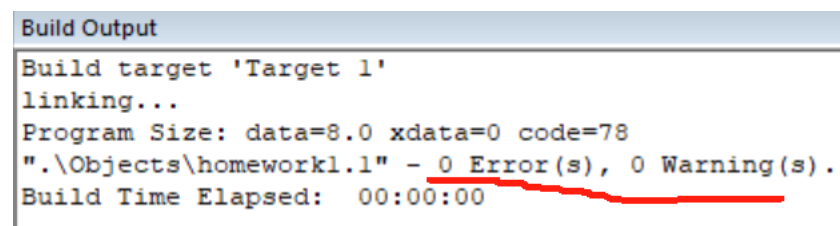
1.1 新建文件，输入汇编程序



```
1  ORG      000H          ;the following saved in the mem start from 000H
2          LJMP     START  ;Long JMP to the label START
3          ORG      040H          ;the following save in the mem start from 040H
4  START:  MOV     SP, #5FH      ;stack pointer initialized
5          MOV     A,  #05H      ;let accumulator A be 05H
6          MOV     B,  #03H      ;let reg B be 03H
7          ADD     A,  B          ;add A $ B, saved in A
8  LOOP:   NOP                ;meaningless
9          LJMP     LOOP        ;Long JMP to the label Loop. Since there's no limits, the LOOP is a dead cycle
10         END;                ;process end
```

如上图，并且添加了断点，程序含义见注释

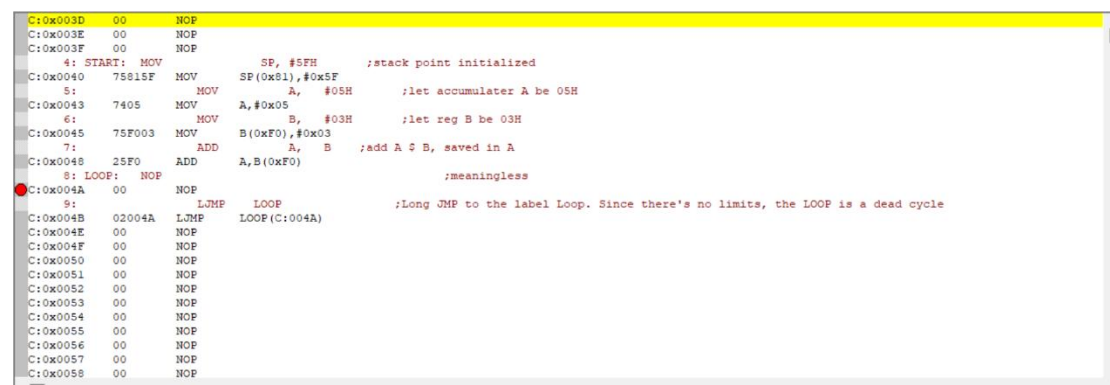
1.2 编译文件



```
Build Output
Build target 'Target 1'
linking...
Program Size: data=8.0 xdata=0 code=78
".\Objects\homework1.1" - 0 Error(s), 0 Warning(s).
Build Time Elapsed: 00:00:00
```

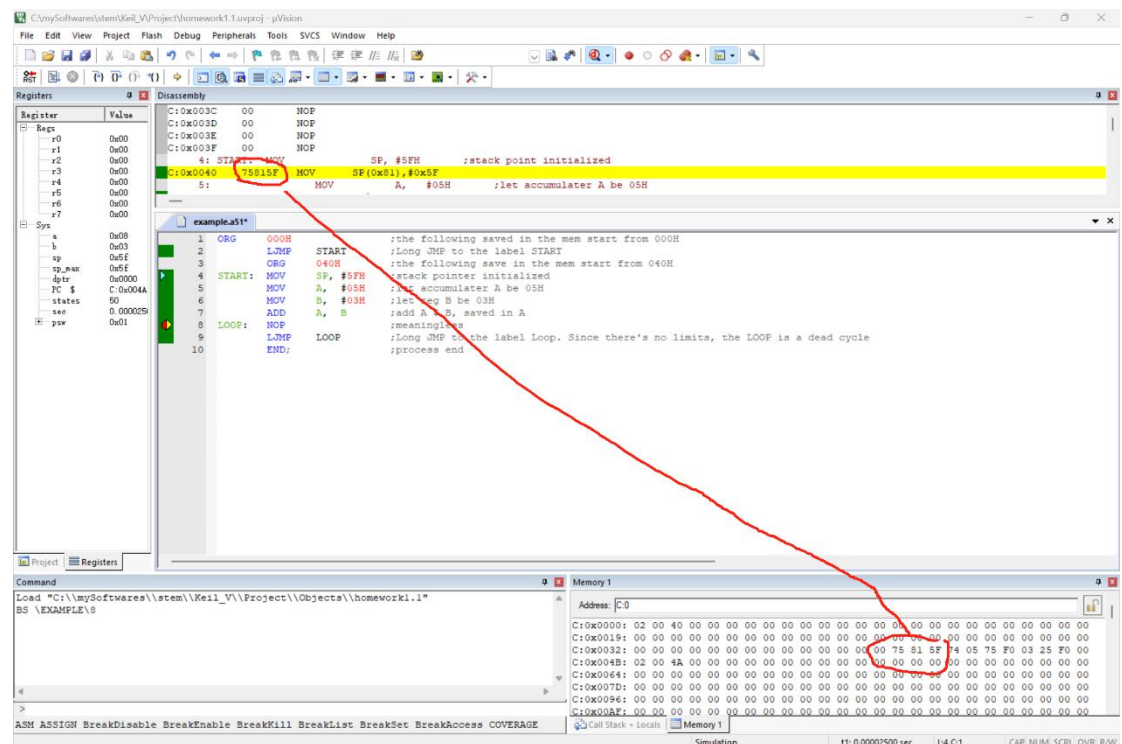
编译成功，没有 Error 和 Warning

1.3 调试程序

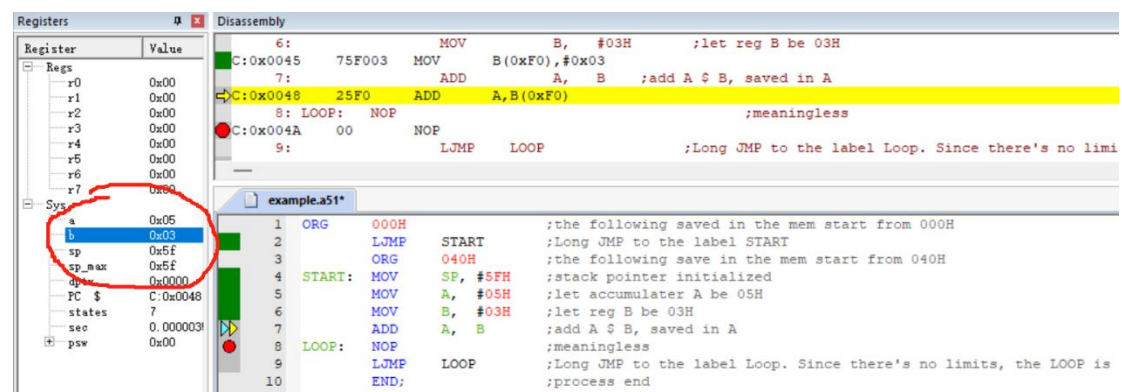


```
C:0x003D 00 NOP
C:0x003E 00 NOP
C:0x003F 00 NOP
4: START: MOV     SP, #5FH      ;stack point initialized
C:0x0040 75815F MOV     SP(0x81),#0x5F
5:          MOV     A,  #05H      ;let accumulator A be 05H
C:0x0043 7405 MOV     A,#0x05
6:          MOV     B,  #03H      ;let reg B be 03H
C:0x0045 75F003 MOV     B(0xF0),#0x03
7:          ADD     A,  B          ;add A $ B, saved in A
C:0x0048 25F0 ADD     A,B(0xF0)
8: LOOP:   NOP                ;meaningless
C:0x004A 00 NOP
9:          LJMP     LOOP        ;Long JMP to the label Loop. Since there's no limits, the LOOP is a dead cycle
C:0x004B 02004A LJMP     LOOP(C:004A)
C:0x004E 00 NOP
C:0x004F 00 NOP
C:0x0050 00 NOP
C:0x0051 00 NOP
C:0x0052 00 NOP
C:0x0053 00 NOP
C:0x0054 00 NOP
C:0x0055 00 NOP
C:0x0056 00 NOP
C:0x0057 00 NOP
C:0x0058 00 NOP
```

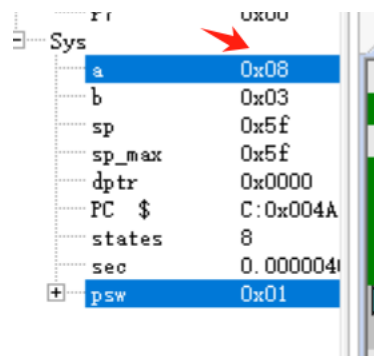
观察上图，可以发现程序在内存中的位置如注释中所示



查看右下角的内存窗口，在对应内存地址保存了指令的机器码



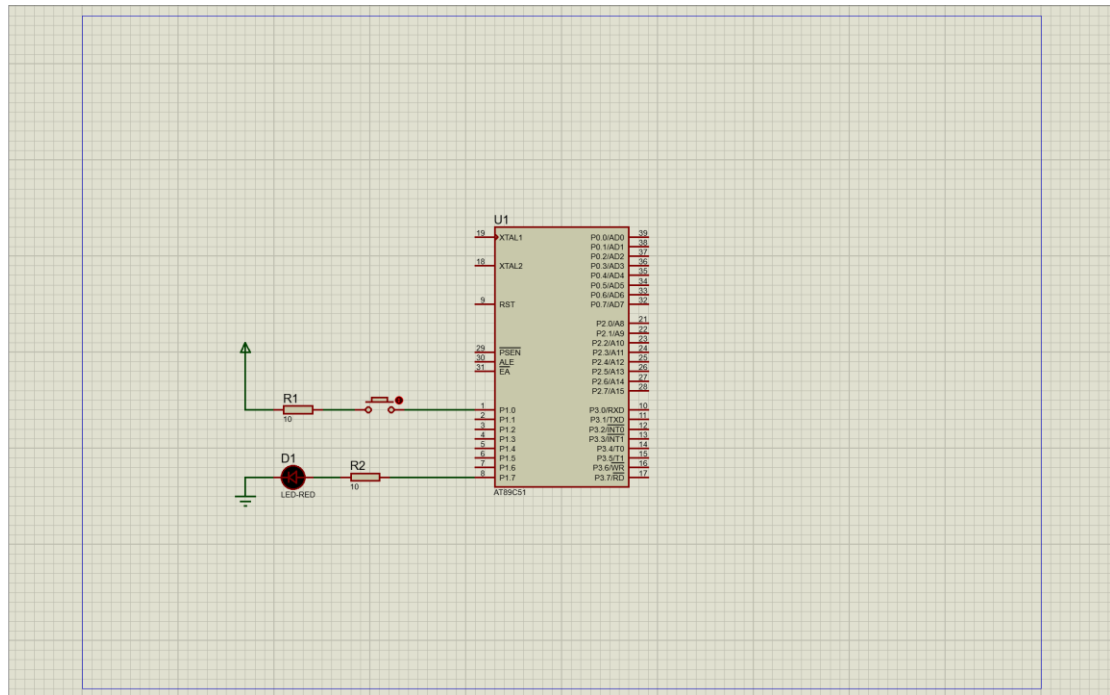
单步运行程序，在所有赋值指令结束后观察寄存器值，发现与设置一致



继续运行，在 ADD 指令运行结束后，累加器 A 中的值变为 05H+03H=08H

2. Keil 与 Proteus 联调使用入门

2.1 连接电路图

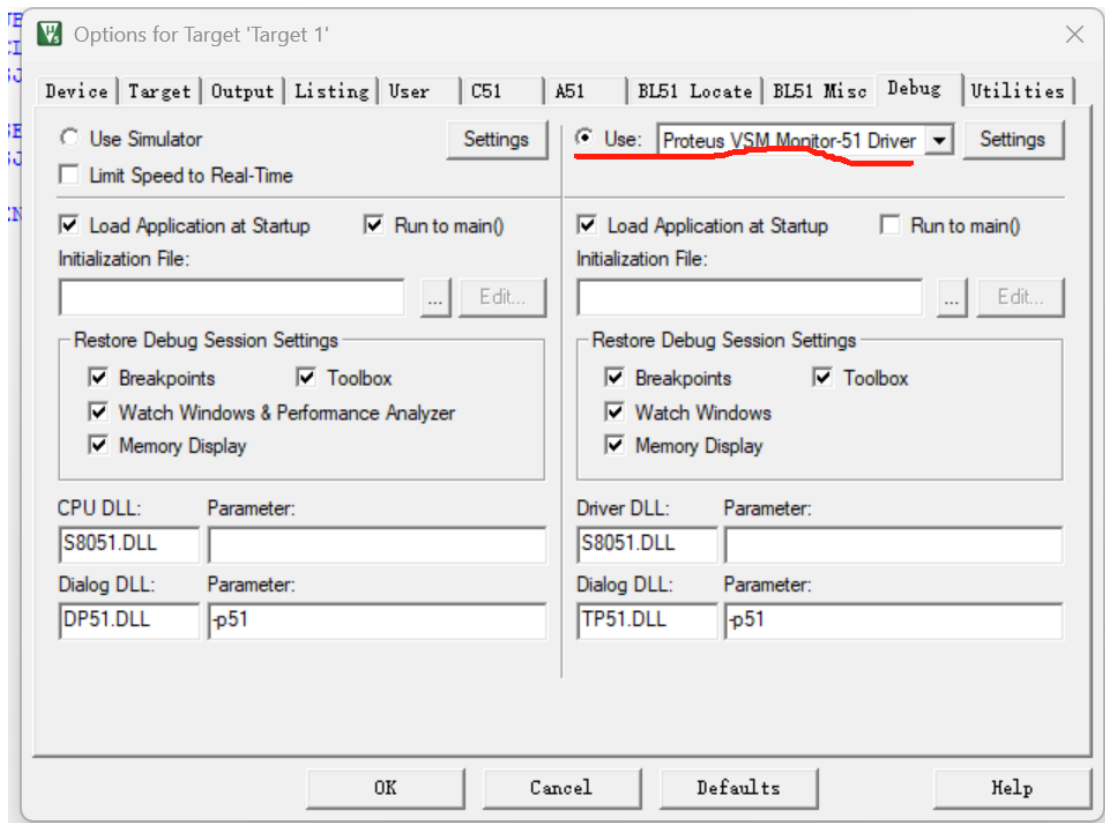
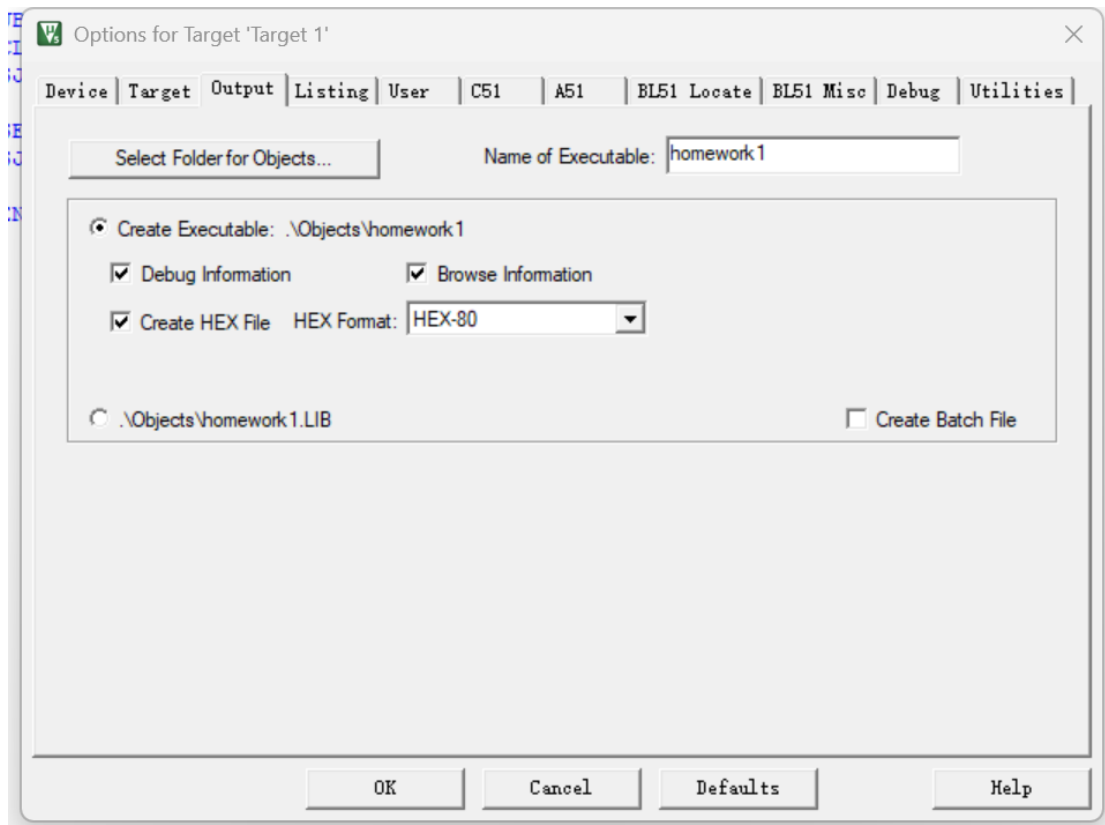


电路图如上所示，注意引脚位置

2.2 根据硬件编写程序

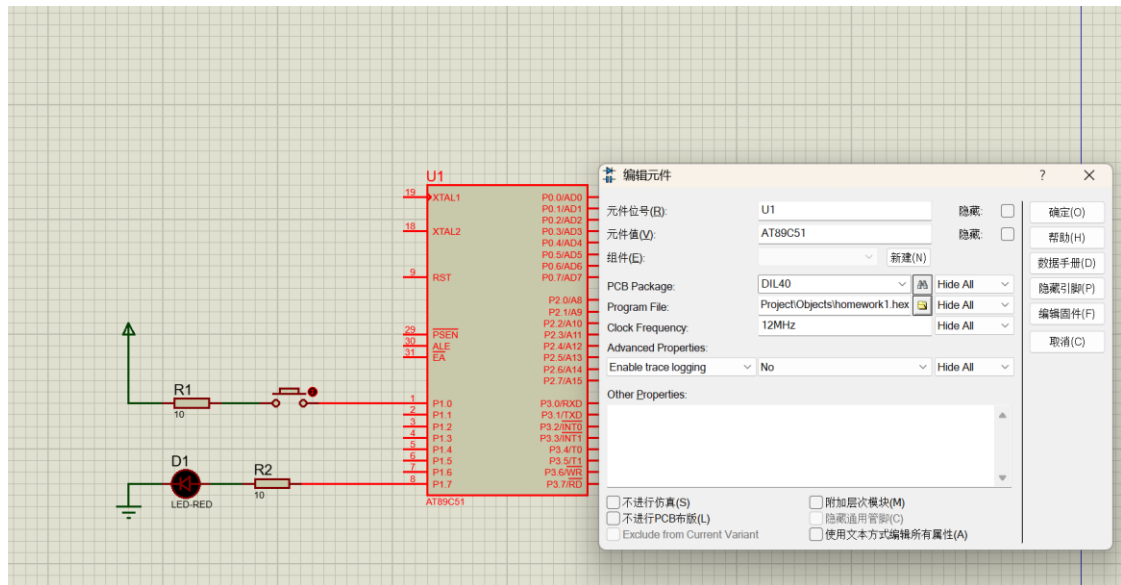
```
test.a51
1      ORG 0000H
2      AJMP START
3      ORG 0003H
4
5  START:      CLR P1.0
6              CLR P1.7
7
8  LIGHT:      JB  P1.0, LIGHT_ON
9              CLR P1.7
10             SJMP LIGHT
11
12 LIGHT_ON:    SETB P1.7
13             SJMP LIGHT
14
15             END
```

程序在一开始清空 P1.0 和 P1.7 的电压，在 LIGHT 块进行持续的 P1.0 电压监控，当 bit is set，那么就 JMP 到 LIGHT_ON，后者将 P1.7 set，二极管导通，灯亮了

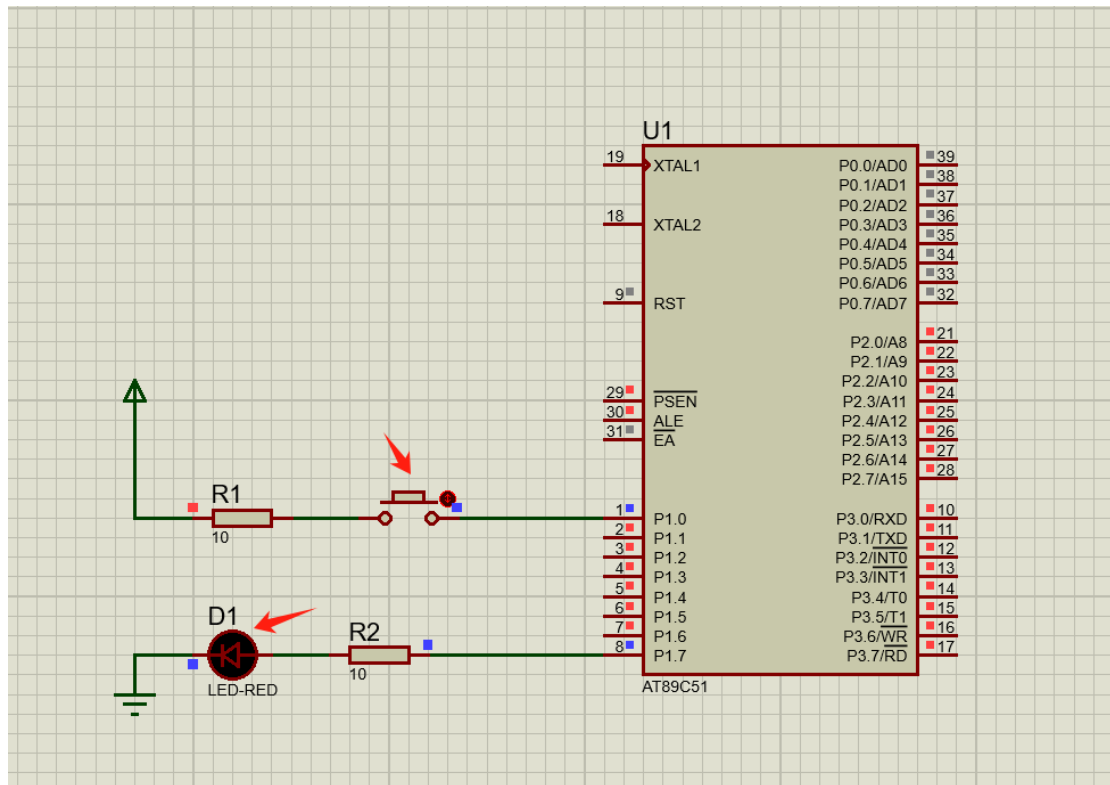


设置联调，输出 hex 文件

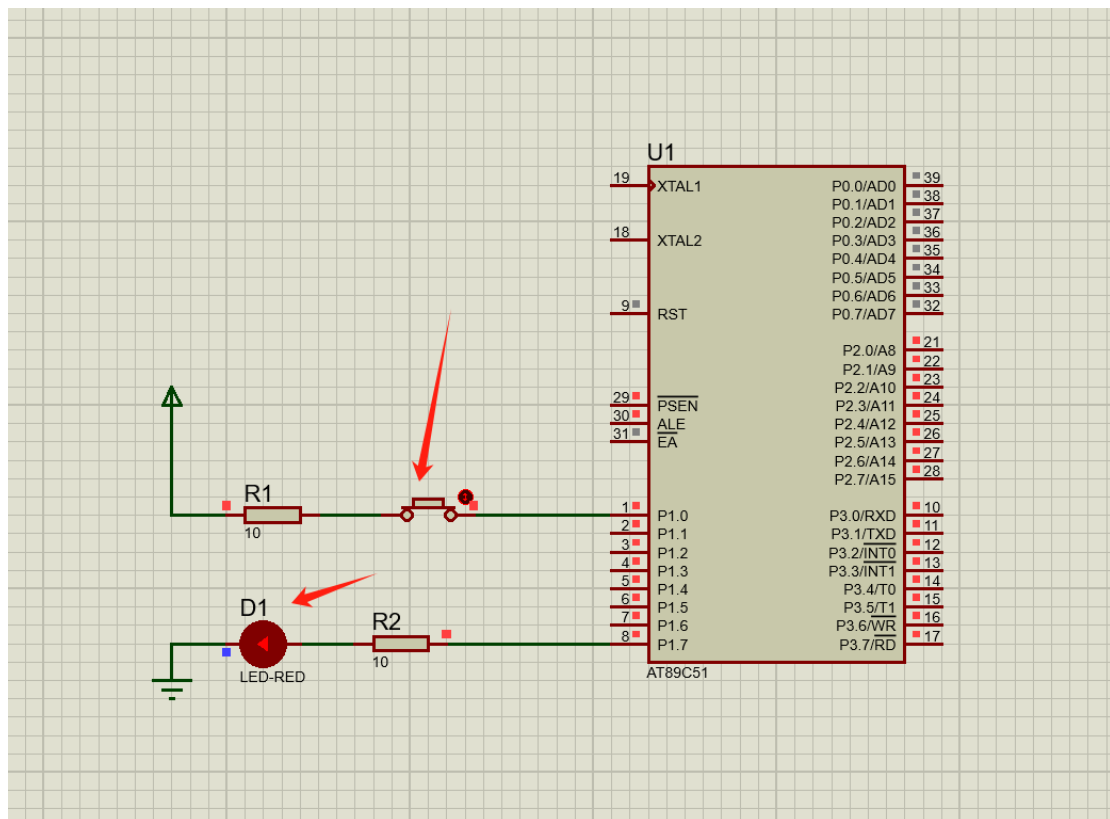
2.3 二者的联调



给单片机导入 HEX 文件



松开按钮，P1.0 是低电平，LED 不亮



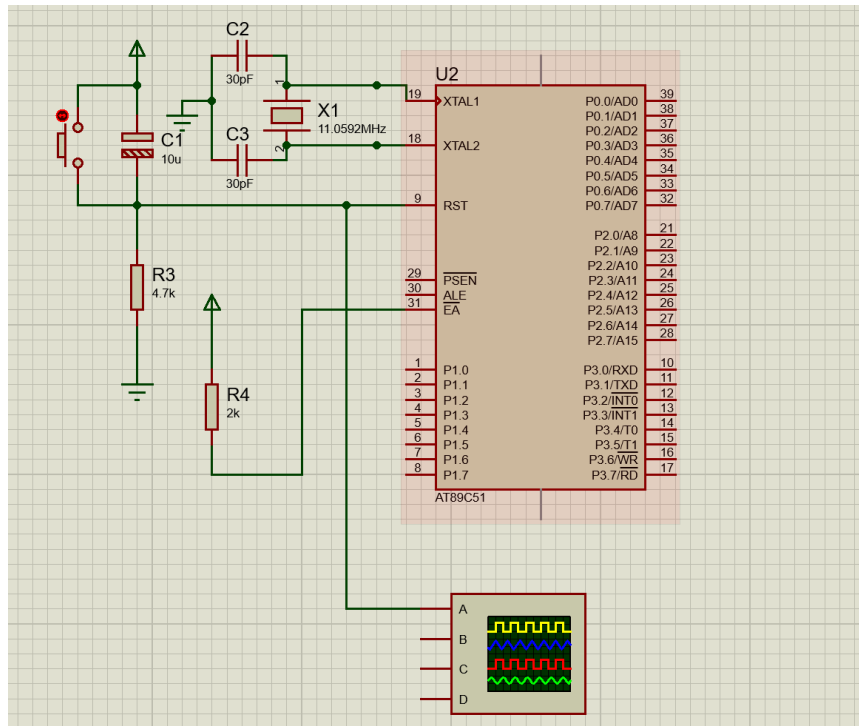
按下按钮，P1.0 变为高电平，程序控制 P1.7 成为高电平，LED 灯点亮

3. MCS-51 的最小系统原理图

最小系统包括了

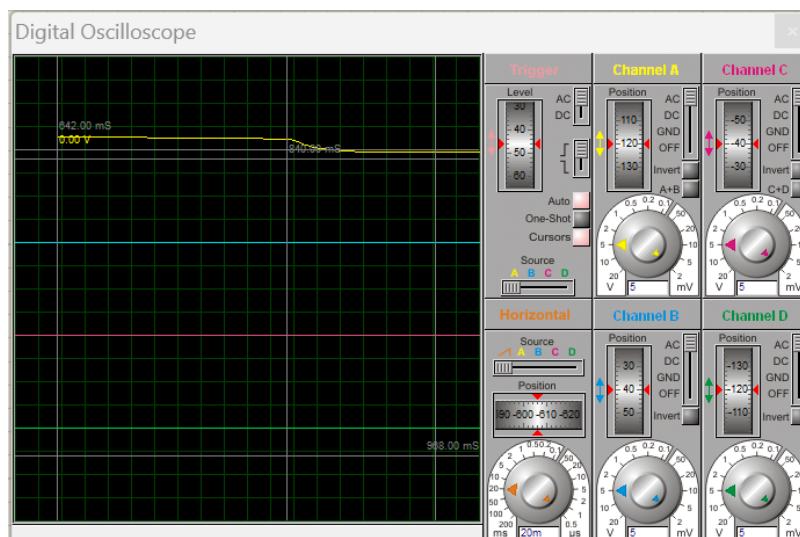
- 电源和地
- 时钟电路
- 复位电路
- 外部 MEM

按键电路原理图如下，设置晶振频率为 11.0592MHz，单片机的 Vcc、GND 由软件自动连接



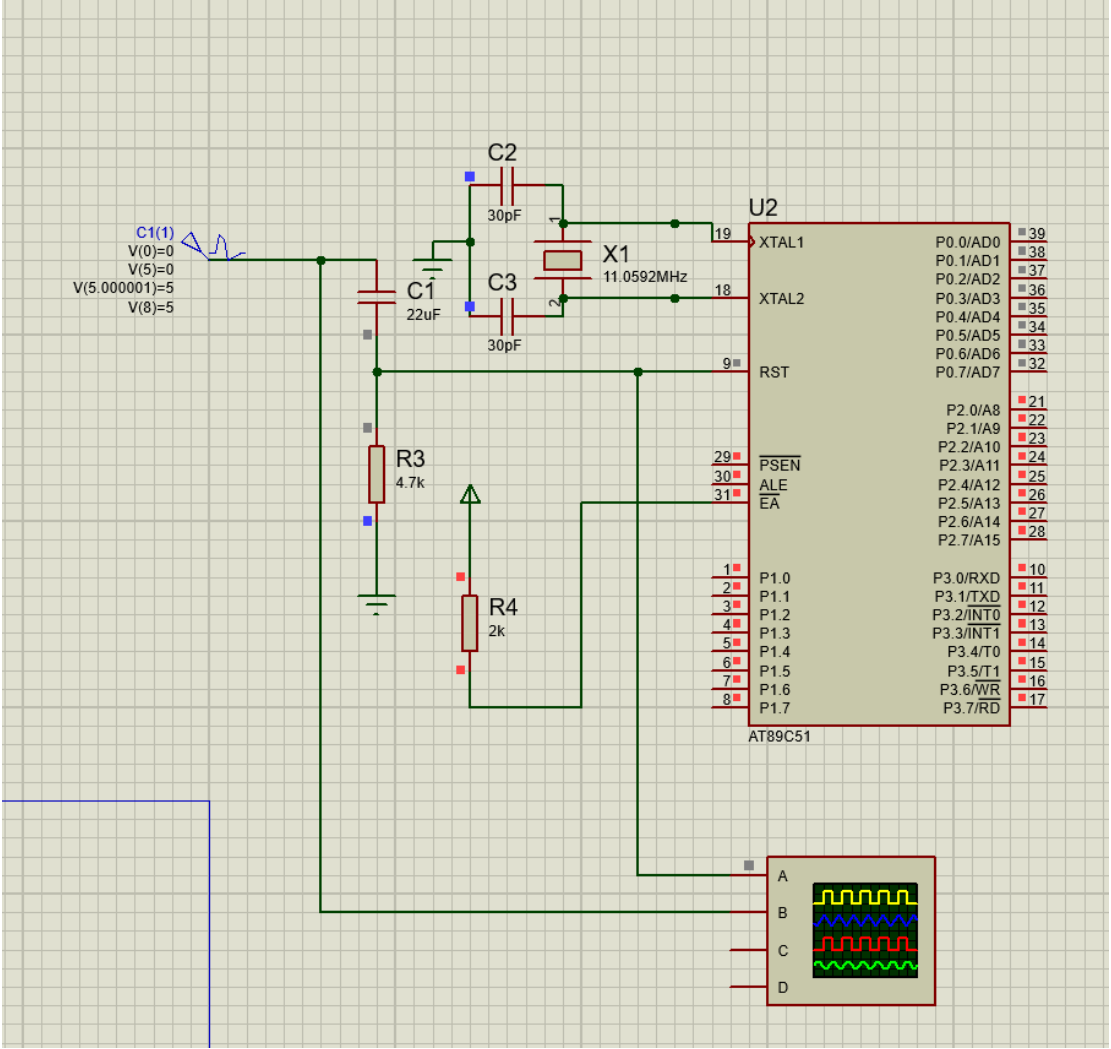
在复位按钮断开时，高电平信号被电容 C1 截断，9 号引脚为低电平；当按下时，电容被按钮短路，高电平信号传导到 9 号引脚上；受 C1-R3 的时间常数影响，9 号引脚的电压呈指数衰减。

9 号引脚在按下按钮后的波形如下

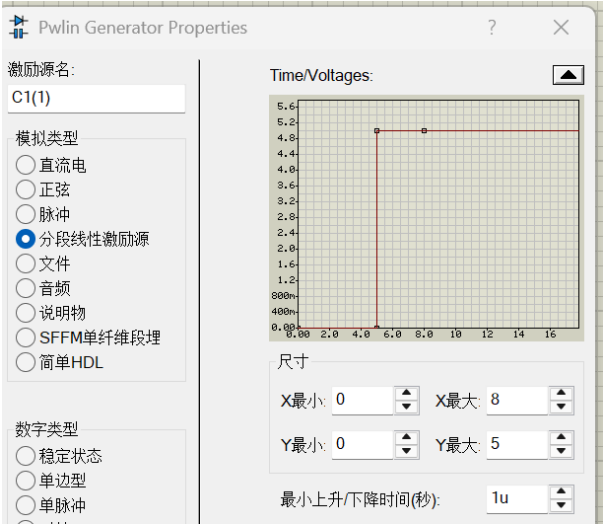


该引脚高电平长达 200ms, 远超要求的 2ms, 可以让系统复位; 其中很大一部分时长是按下按钮的延迟

上电复位原理图如下

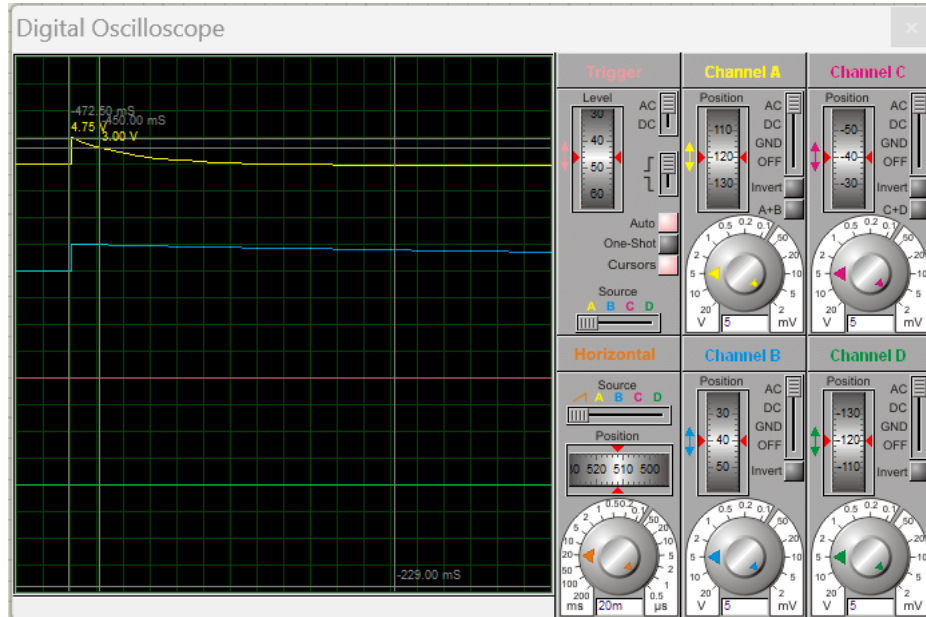


其中信号发生器产生一个阶跃信号，波形如下



在仿真的前 5 秒，复位电路未上电，无法复位，9 号引脚输入低电平

在仿真的第 5 秒，复位电路产生一个幅度为 5V 的阶跃上升，而后指数衰减，结果如下



Channel A 显示 9 号端口输入电压；Channel B 显示信号发生器信号。

结果显示 9 号引脚的高电平长达 20ms，远超要求的 2ms，可以让系统复位